

Programación Multinúcleo y Extensiones SIMD: Implementación y Análisis del Método de Jacobi

BAPTISTE ROBIN, ÁLVARO SCHWIEDOP SOUTO

Arquitectura de Computadores • 2º Grao Enx. Informática

Universidade de Santiago de Compostela

curso1523@usc.es

19 de abril de 2026

Resumen

Se implementan cuatro versiones del método iterativo de Jacobi en C sobre el supercomputador FinisTerra III del CESGA: versión secuencial de referencia (v1), versión optimizada para caché con división de lazos, desenrollamiento y blocking (v2), versión paralela con OpenMP (v3) y versión vectorial con instrucciones AVX256 y FMA (v4). Para $n=3200$, la versión SIMD (v4) alcanza un speedup de $1.45\times$ sobre v2-03, mientras que la mejor configuración OpenMP (v3, *static*, 32 hilos) consigue $25.43\times$ respecto a v3 con 1 hilo. Los experimentos muestran que el escalado OpenMP depende del tamaño: a 32 hilos, v3 pasa de $25.25\times$ en $n=1250$ a $25.43\times$ en $n=3200$.

Palabras clave: Jacobi, AVX256, FMA, OpenMP, cache blocking, speedup, FinisTerra III.

I. INTRODUCCIÓN

El método de Jacobi [1] es un algoritmo iterativo clásico para la resolución de sistemas $Ax=b$ con matriz diagonalmente dominante. Partiendo de $x^{(0)}=0$, en cada iteración se actualiza:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

hasta que $\|x^{(k+1)} - x^{(k)}\|_2 < \text{tol}$. La doble suma implica $\mathcal{O}(n^2)$ operaciones por iteración, lo que hace que tamaños grandes sean costosos y susceptibles de aceleración mediante optimizaciones de bajo nivel.

El objetivo es implementar cuatro versiones de complejidad creciente, medirlas en FinisTerra III (Intel Xeon Platinum 8352Y, 2.2 GHz, 64 núcleos por nodo) y cuantificar el *speedup* obtenido. La Sección II describe las implementaciones; la Sección III, la metodología experimental; la Sección IV, los resultados; la Sección V, las conclusiones.

II. IMPLEMENTACIONES

A. v1 — Secuencial base

Traducción directa del pseudocódigo del enunciado sin ninguna optimización manual. El algoritmo recibe la matriz A , el vector b , la tolerancia tol y el máximo de iteraciones max_iter ; itera actualizando x hasta convergencia:

Entradas: $A \in \mathbb{R}^{n \times n}$, $b, x \in \mathbb{R}^n$, tol , max_iter

Aux: $x_{\text{new}} \in \mathbb{R}^n$

para $\text{iter} = 0$ **hasta** max_iter :

$\text{norm2} \leftarrow 0$

para $i = 0$ **hasta** $n - 1$:

$\sigma \leftarrow 0$

para $j = 0$ **hasta** $n - 1$:

si $i \neq j$: $\sigma \leftarrow \sigma + a[i][j] \cdot x[j]$

$x_{\text{new}}[i] \leftarrow (b[i] - \sigma) / a[i][i]$

$\text{norm2} \leftarrow \text{norm2} + (x_{\text{new}}[i] - x[i])^2$

$x \leftarrow x_{\text{new}}$

si $\sqrt{\text{norm2}} < \text{tol}$: **terminar**

Salida: imprimir norm2

La implementación en C introduce la condición `if (i!=j)` dentro del bucle interno, lo cual genera una rama innecesaria:

```
for (int j = 0; j < n; j++)
    if (i != j)
        sigma += a[i*n+j] * x[j];
x_new[i] = (b[i]-sigma) / a[i*n+i];
```

Se usa como referencia de rendimiento y corrección numérica.

B. v2 — Optimizaciones de caché

Se aplican cuatro mejoras sobre v1:

1. **Reducción de instrucciones:** puntero de fila `const double *row=&a[i*n]`, elimina el recalclo del índice.
2. **División de lazos:** el bucle j se parte en $[0, i)$ e (i, n) , eliminando la rama condicional.
3. **Desenrollamiento $\times 4$:** cuatro acumulaciones explícitas por iteración.
4. **Cache blocking:** bloques de `BLOCK_SIZE=64` filas

para mantener x en caché L1/L2.

C. v3 — OpenMP

La región `parallel` se sitúa *fuera* del bucle de iteraciones para mantener vivos los hilos entre iteraciones y evitar *fork/join* repetido. La variante base usa `reduction(+:norm2)` con `schedule(static)` y, a partir del mismo fuente, se compilan las variantes `dynamic(16)`, `guided` y `critical`. La sincronización entre iteraciones se realiza mediante barreras implícitas y un bloque `omp single`:

```
#pragma omp parallel default(none) \
    shared(a,b,x,x_new,n,norm2,...)
{
    while (...) {
        #pragma omp for schedule(static) \
            reduction(+:norm2)
        for (int i=0; i<n; i++) { ... }
        #pragma omp single
        { converged = sqrt(norm2) < tol; iter++; }
        #pragma omp for schedule(static)
        for (int i=0; i<n; i++) x[i] = x_new[i];
    }
}
```

D. v4 — AVX256 + FMA

Se emplea `__m256d` (4 double/registro) con `__mm256_fmadd_pd`. La memoria se reserva alineada a 32 B con `aligned_alloc` y el *stride* de fila se redondea a múltiplo de 4 (`n_pad`). La reducción horizontal usa `__mm256_hadd_pd`:

```
__m256d va = __mm256_load_pd(&row[j]);
__m256d vx = __mm256_load_pd(&x[j]);
vsigma = __mm256_fmadd_pd(va, vx, vsigma);
/* reduccion horizontal al final */
__m256d h = __mm256_hadd_pd(vsigma, vsigma);
```

No se usan intrínsecos `_u` (no alineados) como exige el enunciado.

III. METODOLOGÍA EXPERIMENTAL

Los experimentos se ejecutan en FinisTerae III mediante trabajos SLURM en la partición `short`. El tiempo se mide en **ciclos de reloj** con `get_counter()` basada en el TSC, englobando sólo el cómputo iterativo. Para cada configuración se toman **10 medidas independientes** y se selecciona la **mediana** para reducir perturbaciones del SO.

Parámetros fijos: `tol=10-5`, `max_iter=15000`, `srand(42)`, dominancia diagonal garantizada sumando la suma de fila al elemento diagonal. Niveles de optimización: `-O0` y `-O3`; v3 usa `-fopenmp`; v4 añade `-mavx2 -mfma`. Hilos evaluados: {1, 2, 4, 8, 16, 32}.

La Tabla 1 resume las medianas obtenidas.

IV. RESULTADOS Y ANÁLISIS

A. Efecto de las optimizaciones de caché

La Figura 1 muestra el *speedup* de v2 respecto a v1 con ambas versiones en `-O0`. Los valores medidos son $n = 1250$: **1.16x**, $n = 2000$: **1.09x** y $n = 3200$: **0.97x**. El efecto depende del tamaño: algunas configuraciones

Cuadro 1: Medianas de ciclos ($\times 10^9$) por versión y tamaño

Versión	$n=1250$	$n=2000$	$n=3200$
v1 -O0	57.3	218.1	887.4
v1 -O3	25.0	98.1	403.4
v2 -O0	49.6	200.1	914.9
v2 -O3	14.4	59.5	384.9
v4 -O0	41.9	160.3	655.6
v4 -O3	14.7	60.7	264.9
v3 -O3 (1T)	24.4	96.3	399.3
v3 -O3 (32T)	1.0	3.3	15.7

mejoran frente a v1 y otras quedan por debajo.

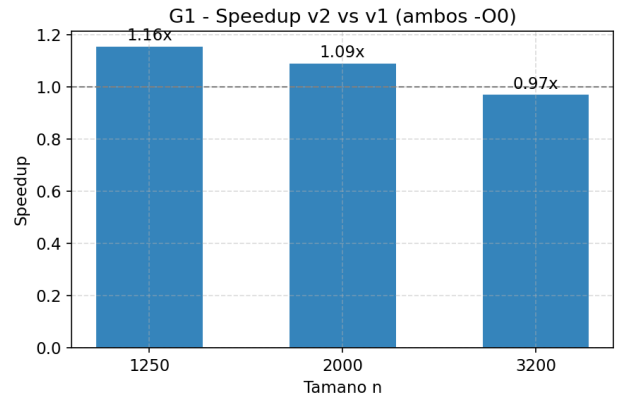


Figura 1: Speedup v2/v1 con `-O0`.

B. Todas las versiones vs. v1-O3

La Figura 2 compara la versión secuencial optimizada (v2), la mejor configuración OpenMP (v3) y la versión SIMD (v4) respecto a v1-O3. Los speedups de v2 son $n = 1250$: **1.73x**, $n = 2000$: **1.65x** y $n = 3200$: **1.05x**; los de v3, $n = 1250$: **25.88x**, $n = 2000$: **29.78x** y $n = 3200$: **25.69x**; y los de v4, $n = 1250$: **1.70x**, $n = 2000$: **1.61x** y $n = 3200$: **1.52x**. En estos resultados, **v3** es la serie más rápida en 3/3 tamaños. Las mejores configuraciones de v3 son $n = 1250$: `static` con 32 hilos, $n = 2000$: `static` con 32 hilos, $n = 3200$: `static` con 32 hilos.

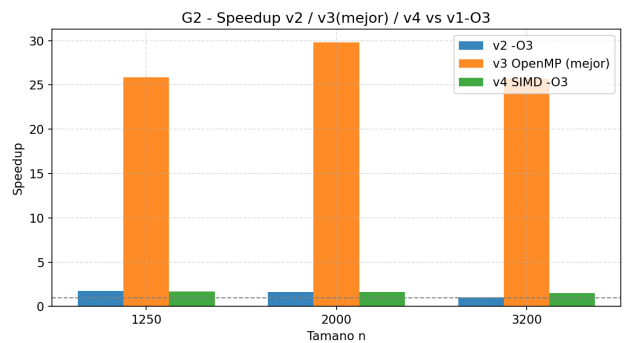


Figura 2: Speedup de v2-O3, la mejor configuración de v3 y v4-O3 respecto a v1-O3.

C. v3 y v4 vs. v2-O3

La Figura 3 muestra el *speedup* de la mejor configuración OpenMP (v3) y de la versión SIMD (v4) respecto a v2-O3. OpenMP obtiene speedups entre $14.93\times$ y $24.51\times$, mientras que SIMD logra entre $0.98\times$ y $1.45\times$. En conjunto, v3 aporta la mayor ganancia respecto a la mejor versión secuencial.

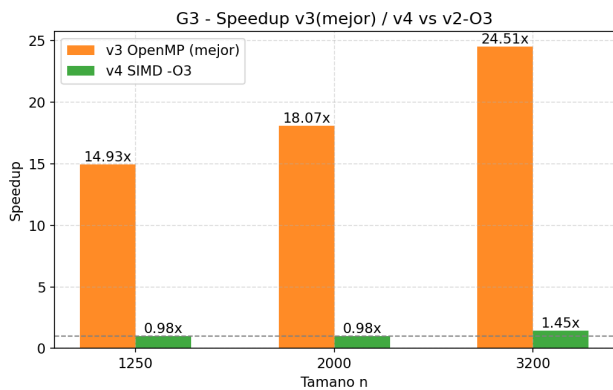


Figura 3: Speedup de la mejor configuración de v3 y de v4-O3 respecto a v2-O3.

D. Escalabilidad de v3 con el número de hilos

La Figura 4 es la gráfica **obligatoria** del enunciado y resume el speedup de v3-O3 con `schedule(static)` frente a 1 hilo. A 32 hilos, los speedups alcanzan $n = 1250$: $25.25\times$, $n = 2000$: $29.25\times$ y $n = 3200$: $25.43\times$. El mejor escalado se observa en $n = 2000$, mientras que $n = 1250$ es el caso más sensible al overhead de sincronización.

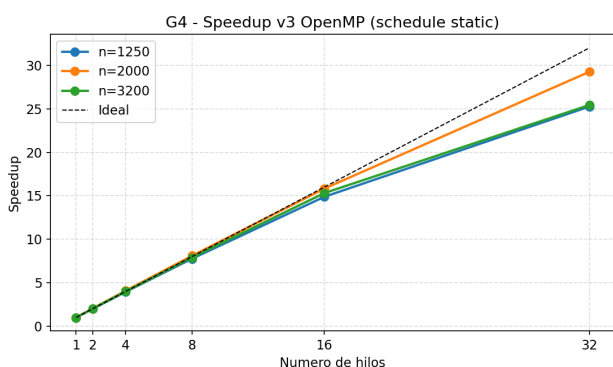


Figura 4: Speedup de v3-O3 (`schedule(static)`) frente a 1 hilo para cada n . Línea discontinua: ideal lineal.

E. Comparación de schedulings

La Figura 5 compara los tres modos de reparto de OpenMP sobre v3. A 32 hilos, la mejor política para cada tamaño es $n = 1250$: static con $25.25\times$, $n = 2000$: static con $29.25\times$, $n = 3200$: static con $25.43\times$.

F. Reducción reduction vs. critical

La Figura 6 compara la variante base con `reduction(+:norm2)` frente a la variante con `critical`. A 32 hilos, se obtiene $n = 1250$: `reduction`= $25.25\times$, `critical`= $21.61\times$; $n = 2000$: `reduction`= $29.25\times$, `critical`= $27.08\times$; $n = 3200$: `reduction`= $25.43\times$, `critical`= $25.24\times$. La mayor diferencia relativa aparece en $n = 1250$, donde `reduction` aventaja a la otra opción en aproximadamente 14.4% .

V. CONCLUSIONES

1. Las optimizaciones de caché de v2 frente a v1 con -O0 producen speedups entre $0.97\times$ y $1.16\times$.
2. La versión SIMD (v4) mejora a v2-O3 con speedups entre $0.98\times$ y $1.45\times$.
3. Con `schedule(static)`, v3 alcanza $n = 1250$: $25.25\times$, $n = 2000$: $29.25\times$ y $n = 3200$: $25.43\times$ a 32 hilos.
4. La mejor política de reparto de OpenMP depende del tamaño del problema, por lo que conviene estudiar static, dynamic(16) y guided de forma separada.
5. La comparación `reduction` vs. `critical` en $n = 3200$ y 32 hilos da $25.43\times$ frente a $25.24\times$.

Como trabajo futuro se propone combinar v3 y v4 (SIMD+OpenMP) y estudiar el impacto del *thread binding* (`OMP_PROC_BIND`) sobre la coherencia de caché en entornos NUMA.

REFERENCIAS

- [1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, Philadelphia, 2003.
- [2] Intel Corporation, *Intel Intrinsics Guide*, <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>, [online] última visita marzo 2026.
- [3] OpenMP Architecture Review Board, *OpenMP API v3.0 Summary Spec*, <https://www.openmp.org/wp-content/uploads/OpenMP3.0-SummarySpec.pdf>, noviembre 2008.
- [4] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design*, 5th ed. Morgan Kaufmann, 2014.

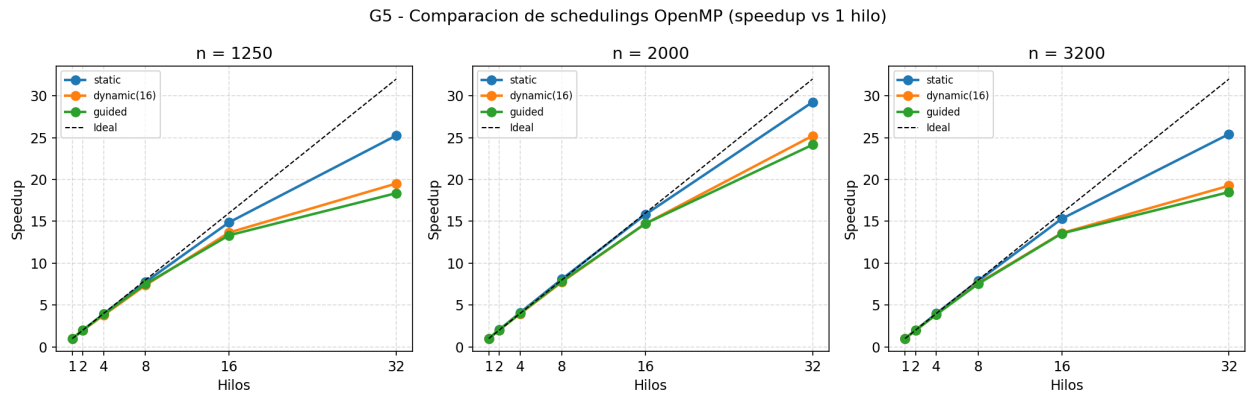


Figura 5: Comparación de schedulings OpenMP para v3 y $n \in \{1250, 2000, 3200\}$.

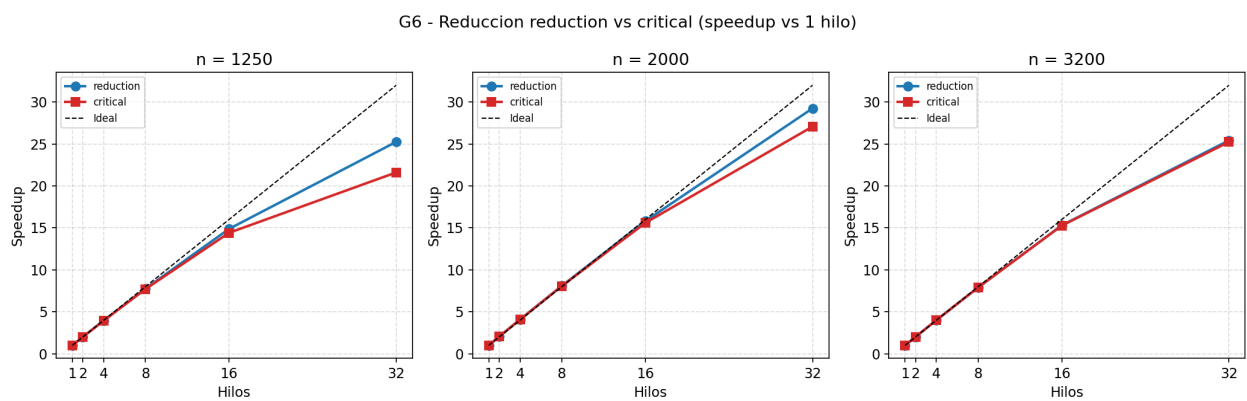


Figura 6: Comparación de reducción *reduction* vs. *critical* en v3.